

AI-SHA Foundation Tutorial

Volume III — CP2 and CP4: the second board, and making the two talk

Volume I built the Jetson (FLASH-CP3). Volume II gave it eyes and ears (CP5, CP6). This volume covers the **other half of the robot**: the Raspberry Pi 5 tier that owns the voice, the drive and the sensors (CP2), and the two-board mesh that connects it to the Jetson (CP4). Same rules as before - every command dodges something that cost us hours, and the coloured boxes say what would otherwise have gone wrong. **Skills are deliberately out of scope here**; they are Volume II and later.

Where you are

FLASH

CP0

CP1

CP2

CP3

CP4

CP5

CP6

CP7-9

THIS TUTORIAL

CP4 reaches a documented limit - read page 7 before planning on it

The traps this tutorial defuses for you

TRAP AVOIDED

Eight detours removed from your path. (1) **The Pi login is the BOARD NAME**, not a person or a project name - three sets of "correct" credentials failed on the username alone. (2) **A Pi 5 that freezes with the network light still on is UNDER-POWERED, not hot** - check `throttled` before you buy a fan. (3) **Copy files OFF the Pi rather than searching on it** - a directory scan is enough load to hang an under-powered board. (4) **Your SSID may not be what you think**: a router can store an HTML-escaped apostrophe *as part of the name*. (5) **Write SSID and password to files** before handing them to `nmccli`. (6) **Both boards need the same ROS_DOMAIN_ID** and nothing warns you when they differ - they simply never see each other. (7) **Never hardcode the domain** in a helper script; it silently breaks the day you change it. (8) **Humble and Jazzy discover each other but do NOT exchange data with real node subscriptions** - plan a bridge, not a reflash.

FROM THE FIELD

Version anchor. Jetson Orin Nano: JetPack 6.2.2 (L4T R36.5.0), Ubuntu 22.04, **ROS 2 Humble**. Raspberry Pi 5 Model B: **Ubuntu 24.04.3, ROS 2 Jazzy**, kernel 6.8. That distro split is not a mistake to be corrected - it is forced by the hardware, and page 7 explains why.

DECISION — WHY THIS VOLUME EXISTS SEPARATELY

Earlier project notes recorded the Pi tier as **"not started - blocked"** and said the robot had **no voice output**. Both were wrong: they were written without ever logging into the board. When we finally did, we found a built ROS 2 workspace, two working text-to-speech engines and a full sensor suite. This volume documents what is actually there, so nobody plans around a gap that does not exist.

CP2

The Raspberry Pi 5 tier - voice, drive, sensors

Runs on: Raspberry Pi 5 Time: ~1 h to survey an existing board

The Pi owns everything the Jetson does not: the **speaker**, the wheels, and the environmental sensors. If you are following this with a board somebody else prepared, your first job is not to install anything - it is to find out what is already there.

1 Find the board, then log in as the BOARD NAME

If the Pi has no DHCP address yet, it is still reachable over IPv6 link-local on the wired interface. Ask every device on the wire to answer, then pick out the Raspberry Pi by its hardware address:

WORKSTATION

```
ping6 -c3 -I eth0 ff02::1          # all-nodes multicast: everyone replies
ip neigh show dev eth0             # 2c:cf:67:* = Raspberry Pi Trading
ssh pi5@fe80::2ecf:67ff:fec6:d08e%eth0 # %iface is REQUIRED for link-local
```

TRAP AVOIDED - THE ONE THAT COST DAYS

The username is the board's hostname. Two sets of credentials we were given (`nvidia` and `aisharjah` , with their passwords) failed for weeks and the board was written off as "needs a reflash". The passwords were *right*; only the username was wrong. This build names the user after the board - the Pi 5 is `pi5` , and an earlier Pi 4 was `pi4` . Before assuming a reflash, try the hostname as the username. Evidence lives in ordinary places: shell history (`ssh pi5@pi5.local`) and Raspberry Pi Imager's saved settings, which record the username it wrote (`sshUserName`) - the password there is a one-way hash and cannot be recovered.

2 Survey what is already installed - gently

PI 5

```
hostname; . /etc/os-release; echo "$PRETTY_NAME"; uname -r
ls /opt/ros/                                # expect: jazzy
ls ~/ros2_ws/src/                           # the packages
ls ~/ros2_ws/install/ | grep -v setup       # what is actually BUILT
lsusb | grep -vi hub                        # LiDAR / Arduino / mic
aplay -l | grep ^card                       # the speaker
```

Note: a package can exist in `install/` but no longer in `src/` - somebody cleaned the source and left the build. It still runs, but you cannot rebuild it. Check both lists, not one.

3 Diagnose the freezes BEFORE you build anything

Our Pi dropped off the network repeatedly - mid-command, roughly a minute after boot. Diagnosing that first saved a lot of wasted integration work:

PI 5

```
vcgencmd measure_temp      # throttling starts near 80C
vcgencmd get_throttled     # 0x0 = never under-volted or throttled
cat /sys/class/net/eth0/carrier # 1 = link still up while the OS is gone
```

TRAP AVOIDED - "IT IS OVERHEATING" USUALLY IS NOT

Ours read **54 C** with `throttled=0x0` - no thermal event at all - yet it locked up under a trivial load (a `find` across the workspace). The giveaway is the **Ethernet carrier staying UP** while the OS stops answering: the PHY has power, the SoC does not. That is a **brownout from an inadequate supply**, not heat. A Pi 5 wants a genuine **5 V / 5 A** USB-C supply; anything less produces exactly these random, unattributable hangs, and people spend days chasing them as software faults.

FROM THE FIELD — WORK AROUND AN UNSTABLE BOARD

Until the supply is fixed, keep the board idle: **copy files off it and read them on the workstation** rather than grepping on the Pi. `scp` barely touches the CPU; a recursive search is enough to hang it. This one habit turned a board we could not survey into one we could.

4 Know what the tier gives you

On the reference build, `~/ros2_ws` held **17 built packages**:

Group	Packages
Voice (the point)	<code>tts_speaker</code> (Piper, local, free, ~0.5 s) and <code>tts_elevenlabs</code> (cloud, higher quality, needs an API key), plus <code>speaker_monitor</code> . Run only ONE TTS node at a time.
Drive	<code>mecanum_driver</code> , <code>motor_control</code> , <code>linear_actuator</code>
Sensors	<code>bno055_imu</code> , <code>neo6m_gps</code> , <code>bmp180_pressure</code> , <code>dht11_sensor</code> , <code>rain_sensor</code> , <code>soil_moisture</code> , with <code>sensors_bringup/launch/sensors.launch.py</code>
Display / other	<code>llm_display</code> , <code>robot_display</code> , <code>esp32cam</code> , <code>plant_disease_bottom</code>

Note: SLAM and navigation (`slam-toolbox`, `robot-localization`, `rf2o`) were **not** present. The LiDAR is wired (a CP210x USB-UART bridge appears in `lsusb`), but turning it into a map is still to do.

5 Give the robot a voice (Piper, local)

Piper needs no network and no API key, which makes it the right thing to prove first. The node expects the binary and the voice model at fixed paths:

PI 5

```
ls ~/.local/bin/piper           # the binary
ls ~/piper_models/en_US-lessac-medium.onnx # the voice

source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
export ROS_DOMAIN_ID=42
ros2 run tts_speaker tts_speaker_node
```

Note: the node subscribes `/tts_text` and publishes `/robot/speaking` and `/speaker/playing`. That last one is the feedback guard - the Jetson's speech recogniser mutes its microphone while the speaker is playing, so the robot does not transcribe its own voice.

6 Make it speak, without any of the rest

PI 5

```
ros2 topic pub --once /tts_text std_msgs/msg/String "{data: 'Hello, I am AI-SHA'}"
aplay -l | grep -i max98357 # the I2S amplifier, usually card 0
```

TRAP AVOIDED

Check a speaker is physically connected before debugging silence. We spent a stretch diagnosing a "TTS problem" on a rig with **no speaker plugged in**. The node was fine. `aplay -l` lists the *amplifier* whether or not a driver is attached to it, so the hardware list is not proof of sound - your ears are.

GATE - proceed only when:

- ☐ You can SSH in with the board-name user and `/opt/ros/jazzy` exists.
- ☐ `ls ~/ros2_ws/install` lists the packages you expect, including a TTS node.
- ☐ `vcgencmd get_throttled` reads `0x0` AND the board survives several minutes of light work without dropping off the network.
- ☐ A message published on `/tts_text` is **audible** from the speaker.

CP4

The two-board mesh - making them see each other

Runs on: both boards Time: ~1 h Read page 7 before relying on it

CP4 is where the Jetson's answers reach the Pi's speaker. Every guide starts at `ROS_DOMAIN_ID` - but there is a step before it that nobody writes down, and it is the one that actually blocks people.

1 First: give the boards a route to each other

Both of our boards were reachable from the workstation, which made it *look* like they were networked. They were not: the Jetson was on a USB device-mode link and the Pi on wired Ethernet - two separate segments that both ended at the workstation. DDS cannot discover across that. Check honestly, from one board to the other:

BOTH BOARDS

```
# from the JETSON, ping the PI's address - not the workstation's
ping -c3 <pi-address>
```

Note: "I can SSH to both" is not the test. The test is whether **board A can reach board B**. Ours could not, and no amount of domain configuration would have helped.

2 Put both boards on the same wireless network

BOTH BOARDS

```
# write the credentials to files first - see the trap below
printf '%s' 'YOUR_SSID' > /tmp/ssid.txt
printf '%s' 'YOUR_PASSWORD' > /tmp/psk.txt
sudo nmcli dev wifi connect "$(cat /tmp/ssid.txt)" \
    password "$(cat /tmp/psk.txt)" ifname <wifi-iface>
rm -f /tmp/psk.txt
ip -4 addr show <wifi-iface> # both must land on the SAME subnet
```

Note: the Wi-Fi interface is `wlP1p1s0` on the Orin Nano and `wlan0` on the Pi. Joining Wi-Fi is additive - the USB link and the wired link keep working, so you do not lose your way in.

TRAP AVOIDED - THE SSID MAY NOT BE WHAT YOU SEE

`nmcli dev wifi connect` failed with **"No network with SSID found"** for a network that was plainly in the scan list. The router had stored an apostrophe **HTML-escaped**, and that escaped text is the broadcast name: `Daou's`, not `Daou's`. **Copy the SSID from the scan output verbatim**. And pass both SSID and password through files - an SSID containing quotes and a password containing `$` will otherwise be mangled by the shell before `nmcli` ever sees them.

3 Put both boards on the same ROS domain

The reference build uses **domain 42**. Set it everywhere the robot starts nodes - service files, launch wrappers, shell profiles - not just in your interactive shell:

BOTH BOARDS

```
export ROS_DOMAIN_ID=42
```

TRAP AVOIDED - NEVER HARDCODE THE DOMAIN IN A HELPER

Changing the stack to domain 42 broke the robot's face-authentication, which had nothing to do with the mesh. The console launched the auth helper with the domain **hardcoded to the old value**, so that one process kept listening on the old domain, saw **no camera at all**, and reported *"I couldn't see a face"* while the user sat perfectly framed in the preview. Helpers must **inherit** the domain from their parent. The same lesson applies to any pinned constant - we hit an identical failure with a camera-rotation value that outlived the mounting it described.

4 Verify discovery in BOTH directions

BOTH BOARDS

```
# on board A
ros2 topic pub -r 2 /mesh_probe std_msgs/String "{data: hello}"

# on board B - name the TYPE explicitly, see the note
ros2 topic list | grep mesh_probe
ros2 topic echo /mesh_probe std_msgs/msg/String --once
```

Note: start the publisher **before** the listener. A subscriber that opens first has no publisher to learn the type from, and reports "does not appear to be published yet" even though the topic is about to exist.

FROM THE FIELD — WHAT A HEALTHY MESH LOOKS LIKE

After joining the network and matching domains, the Pi could list the Jetson's entire graph - `/robot_speech`, `/admin_response`, `/aisha/mode` and every `/camera/**` topic - and `ros2 topic info` reported the **same publisher and subscription counts from both boards**. Discovery across the two machines was complete. That is the good news; page 7 is the qualification.

5 Understand the limit before you design around it

TRAP AVOIDED - DISCOVERY IS NOT DELIVERY

Our two boards run **different ROS 2 distributions** - Humble on the Jetson, Jazzy on the Pi - and the result is genuinely confusing: **discovery works, data does not**.

Works: `ros2 topic list` and `ros2 topic info` agree from both sides; a CLI `ros2 topic pub` on one board reaches a CLI `ros2 topic echo` on the other - *provided you name the message type explicitly*.

Fails: an ordinary node subscription (a normal `create_subscription` in your own program) receives **nothing**, while running perfectly happily.

The tell is the one that looks harmless: on the newer distro, `ros2 topic echo` without a type says **"Could not determine the type"**. Newer ROS 2 releases publish a **type hash** with each endpoint that the older release never sends, so real subscriptions never match. You may also see `sequence size exceeds remaining buffer`.

DECISION — BRIDGE, DO NOT REFLASH

The instinct is to make both boards run the same distribution. **On this hardware you cannot**. The Pi runs Ubuntu 24.04, and the older distro has no packages for it; the Jetson's JetPack pins it to Ubuntu 22.04 with CUDA and TensorRT built against that, so it cannot move forward either. Matching them means **reflashing the Pi and rebuilding a working tier from scratch** - a large, risky job to move a few short strings.

Look at what actually crosses the boards: **answer text and two boolean flags**. So bridge it:

- **A small TCP bridge** - one script per board, each subscribing and publishing only on its OWN distro, passing text between them over a socket. No DDS crossing, immune to distro drift, and trivial to debug. This is the recommended route.
- **A DDS router** (e.g. eProsima DDS Router) - heavier, but it forwards arbitrary topics and is the right answer if you later need shared transforms, LiDAR scans or navigation goals across the two boards.

Reflashing only makes sense if you are rebuilding the Pi tier anyway.

GATE - proceed only when:

- ☐ Board A can **ping board B directly** - not merely "both reachable from my laptop".
- ☐ Both boards report the same `ROS_DOMAIN_ID`, including inside services and launch wrappers, with no hardcoded copies left behind.
- ☐ `ros2 topic list` on each board shows the other board's topics.
- ☐ You have decided how application data will cross: **same distro** (native DDS), or a **bridge** if the distros differ - and you have tested it with a real node subscription, not just `ros2 topic echo`.

Both checkpoints are real, but neither is finished. This page is the honest remainder so nobody plans on a capability that is not there yet.

THE BLOCKER THAT GATES EVERYTHING ELSE - POWER

The Pi froze repeatedly under trivial load, with the network carrier still up and no thermal event recorded. Until it runs from a genuine **5 V / 5 A** supply, every integration session will be interrupted mid-task and you will misattribute the failures to software.

Fix the supply before building on this tier. It is the cheapest item on this page and it blocks the rest.

Open item	What is needed
The audible loop	The Jetson answers and the Pi can speak, but the two halves are not yet joined by a working data path - see the bridge decision on page 7. A speaker must also be physically connected; ours was not.
Navigation / SLAM	The LiDAR is wired and detected, but <code>slam-toolbox</code> , <code>robot-localization</code> and <code>rf2o</code> are not installed. Mapping and autonomous movement are genuinely new work, not configuration.
Static addressing	Both boards currently take DHCP leases. The moment anything references an address, a re-lease will silently break it. CP0's static-IP step becomes real here.
Sensor bring-up	The sensor packages are built but were not exercised in this pass; <code>sensors.launch.py</code> is the entry point.

FROM THE FIELD — THE HABIT THAT SAVED THE MOST TIME

Three separate failures in this volume - the login, the freezes, and the face gate - were each misdiagnosed at first because the **symptom named the wrong layer**. "Wrong password" was a wrong username. "Overheating" was under-power. "Cannot recognise the face" was a process listening on the wrong ROS domain. When a symptom points at a component, **measure that component directly** before changing it: read the temperature, print the environment, dump a frame from the camera. Every one of those measurements took under a minute and each overturned the obvious explanation.